



UNIVERSIDADE FEDERAL DO MARANHÃO
BACHARELADO INTERDISCIPLINAR EM CIÊNCIA E TECNOLOGIA

Relatório de Execução

SmartLight
Semáforo Inteligente

Responsáveis:

Alana Mayara Silva Monteiro

Arthur Felipe Mourão de Oliveira

João Guilherme Oliveira marques

Lara Sabrina da Silva Costa

Período de Execução:

16 de Março de 2026 até 22 de Junho de 2026.

São Luís - MA

1 - Escopo Técnico

O objetivo deste projeto é desenvolver um protótipo funcional de semáforo inteligente utilizando o microcontrolador Arduino estruturado por meio de uma Máquina de Estados Finitos (FSM), integrado a um painel web para monitoramento do tráfego e dos estados lógicos em tempo real.

Hardware e Conectividade: Utilização de um Arduino como núcleo de processamento, três LEDs (Verde, Amarelo e Vermelho) com resistores limitadores para sinalização visual, um Sensor de Presença (PIR) para detectar a aproximação de veículos, e um Botão (Push-Button) que atua como gatilho de ativação do sistema. Todos os componentes estão interconectados via protoboard e jumpers.

Lógica de Operação: O sistema opera sob demanda. Sem o acionamento do botão, o semáforo permanece indefinidamente no estado Verde e o sensor PIR mantém-se desativado (sem detectar movimentos). A partir do momento em que o botão é pressionado, o sensor é habilitado e valida a presença de fluxo. Caso o sensor reconheça um novo movimento após a ativação inicial, a contagem de tempo do estado atual é reiniciada (reset), garantindo dinamismo ao fluxo de tráfego.

Interface de Monitoramento (Web): Criação de um Frontend em HTML, CSS e JavaScript que consome uma API serial via porta USB COM. Esta interface atualiza, em tempo real, um painel visual que reflete tanto o fluxo de veículos e o estado exato (Verde, Amarelo, Vermelho) do semáforo físico, quanto os eventos de reinicialização da contagem disparados pelo sensor.

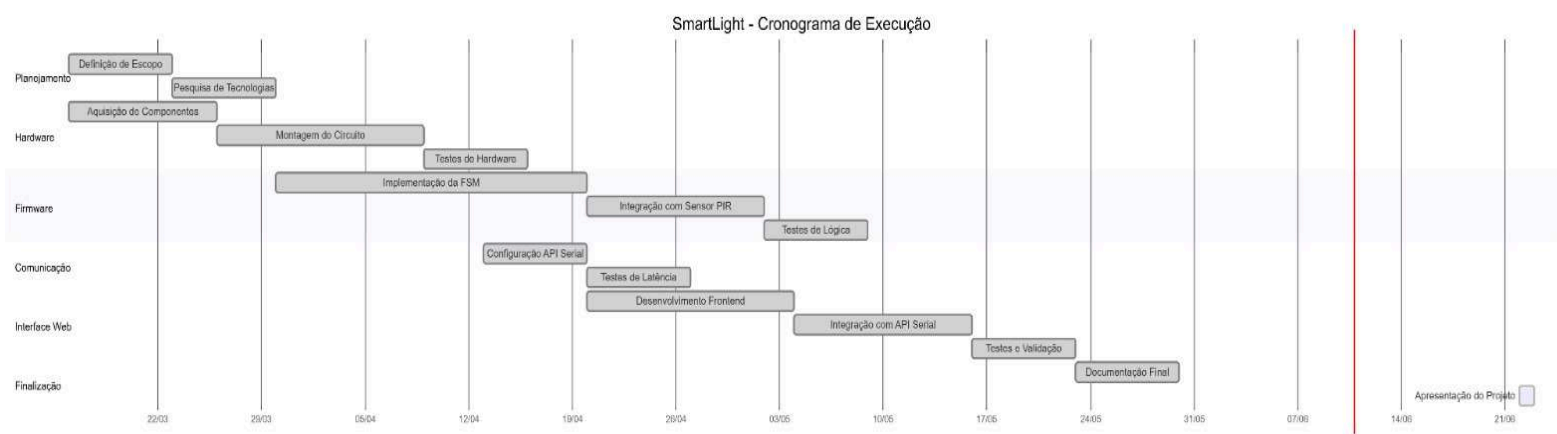
2 - Procedimento de Execução

1. **Montagem do Circuito:** Conexão do sensor PIR, do botão push e dos LEDs aos pinos digitais do Arduino conforme o esquema elétrico, garantindo o correto aterramento e proteção dos componentes.
2. **Codificação da FSM:** Implementação do código de controle em C/C++, estabelecendo a condição inicial de repouso (Semáforo em Verde / Sensor inativo). A lógica da FSM foi estruturada para transicionar de estado apenas após o estímulo do botão e para reiniciar os temporizadores internos sempre que o sensor PIR capturar um novo movimento consecutivamente.
3. **Estabelecimento do Link Serial:** Configuração da comunicação bidirecional utilizando a API serial via porta USB COM para escuta direta dos eventos de

transição de estado e reinício de contagem, substituindo qualquer necessidade de um servidor intermediário.

4. **Integração Web:** Testes de latência entre o acionamento físico do botão, as detecções subsequentes do sensor (resets de tempo) e a imediata atualização do painel de visualização web através dos dados trafegados pela API serial. .

3 - Desenvolvimento/Cronograma de Execução



4 - Resultados

O projeto SmartLight apresentou uma solução robusta e adaptativa para a gestão urbana. A apresentação foca em três pilares: a estabilidade da montagem física com Arduino; a precisão da lógica de estados finitos (FSM); e a eficiência da comunicação direta via API serial via COM.

Os testes confirmaram o funcionamento adequado do protótipo. A implementação da FSM garantiu transições corretas entre os sinais, validando com sucesso o comportamento de repouso (semáforo travado no Verde com sensor desabilitado) até que o botão push fosse acionado. Da mesma forma, o mecanismo de segurança que reinicia a contagem de tempo ao detectar novos movimentos sucessivos respondeu perfeitamente no hardware.

Ademais, destaca-se que a interface web também já está pronta e totalmente funcional; a comunicação via API serial USB COM provou-se altamente eficiente, espelhando instantaneamente no painel digital cada mudança de estado e cada reinício de contagem gerado no mundo físico. O protótipo demonstra que é possível gerir infraestruturas de tráfego de forma inteligente, utilizando dados em tempo real para otimizar o fluxo, servindo como uma prova de conceito escalável para cidades inteligentes.

5 - Desafios Técnicos e Adaptações do Escopo

Durante a fase de desenvolvimento, o planejamento inicial previa a inclusão do estado Amarelo Piscante no protótipo físico, uma funcionalidade comumente utilizada em semáforos reais para indicar períodos de baixo tráfego ou estados de alerta. No entanto, devido à complexidade de sincronização temporal concorrente na Máquina de Estados Finitos (FSM) do Arduino — especialmente ao conciliar as interrupções de leitura do botão push e os resets do sensor PIR —, a implementação dessa rotina no hardware físico não foi viável no tempo hábil do projeto.

Para contornar essa limitação sem prejudicar a proposta de valor do sistema, o recurso foi adaptado e implementado exclusivamente no ambiente virtual como uma simulação. A interface web foi programada para simular perfeitamente o comportamento do "Amarelo Piscante" de forma isolada, demonstrando como a lógica se comportaria em um cenário de implantação real. Essa separação permitiu manter a estabilidade do hardware focado na dinâmica sob demanda (botão/sensor), enquanto o painel web validou a viabilidade visual e lógica da funcionalidade para futuras atualizações do sistema físico.

6 - Anexos

sketch_may12a.ino

```
1  const int pinoVerde = 8;
2  const int pinoAmarelo = 9;
3  const int pinoVermelho = 10;
4  const int pinoPIR = 2;
5
6  void setup() {
7      pinMode(pinoVerde, OUTPUT);
8      pinMode(pinoAmarelo, OUTPUT);
9      pinMode(pinoVermelho, OUTPUT);
10     pinMode(pinoPIR, INPUT);
11
12     Serial.begin(9600);
13
14     // Estado inicial
15     digitalWrite(pinoVerde, HIGH);
16     digitalWrite(pinoAmarelo, LOW);
17     digitalWrite(pinoVermelho, LOW);
18
19     Serial.println("Sistema Iniciado. Estado: VERDE");
20     delay(1000);
21 }
22
23 void loop() {
24     int estadoSensor = digitalRead(pinoPIR);
25
26     if (estadoSensor == HIGH) {
27         Serial.println("\n[!] Movimento detectado!");
28
29         // 1. Sinal Amarelo
30         digitalWrite(pinoVerde, LOW);
31         digitalWrite(pinoAmarelo, HIGH);
32         Serial.println("Estado: AMARELO (Atenção)");
33         delay(4000);
34
35         // 2. Sinal Vermelho com Contador
36         digitalWrite(pinoAmarelo, LOW);
37         digitalWrite(pinoVermelho, HIGH);
38         Serial.println("Estado: VERMELHO (Pare)");
39
40         // Contador regressivo de 8 segundos
41         for (int i = 8; i > 0; i--) {
42             Serial.print("Tempo restante: ");
43             Serial.print(i);
44             Serial.println("s");
45             delay(1000); // Espera 1 segundo a cada ciclo
46         }
47     }
```

```

// 3. Retorna para o Verde
digitalWrite(pinoVermelho, LOW);
digitalWrite(pinoVerde, HIGH);
Serial.println("Estado: VERDE (Siga)");

// Delay de segurança para evitar reativação imediata
delay(2000);
}

delay(100);

```

```

- -
Estado: AMARELO (Atenção)
Estado: VERMELHO (Pare)
Tempo restante: 8s
Tempo restante: 7s
Tempo restante: 6s
Tempo restante: 5s
Tempo restante: 4s
Tempo restante: 3s
Tempo restante: 2s
Tempo restante: 1s
Estado: VERDE (Siga)

```

```

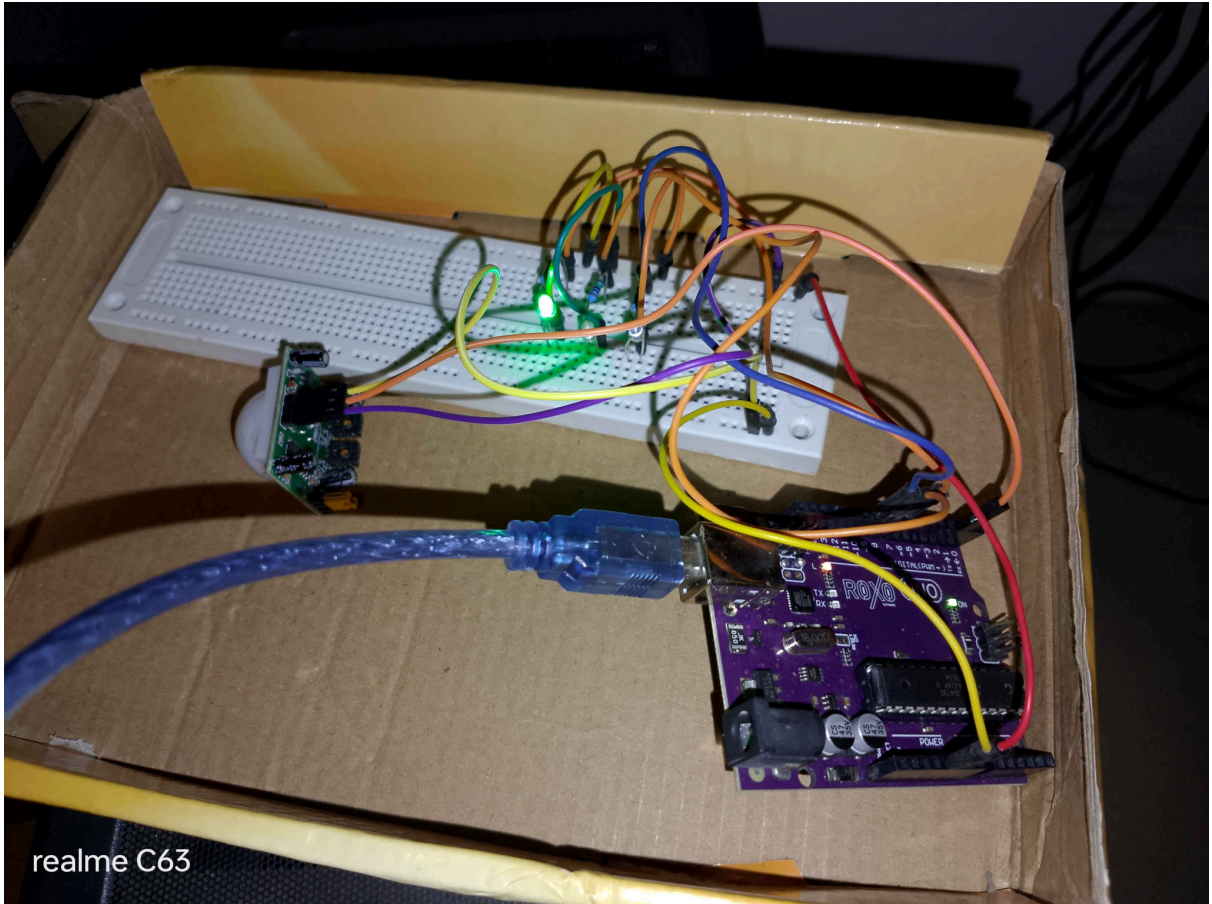
[!] Movimento detectado!
Estado: AMARELO (Atenção)
Estado: VERMELHO (Pare)
Tempo restante: 8s
Tempo restante: 7s
Tempo restante: 6s
Tempo restante: 5s
Tempo restante: 4s
Tempo restante: 3s
Tempo restante: 2s
Tempo restante: 1s
Estado: VERDE (Siga)

```

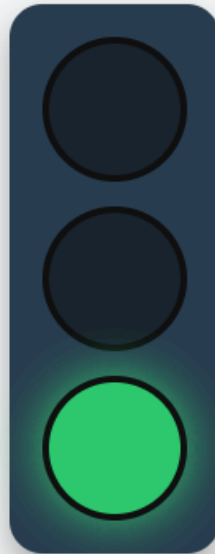
```

[!] Movimento detectado!
Estado: AMARELO (Atenção)
Estado: VERMELHO (Pare)
Tempo restante: 8s
Tempo restante: 7s
Tempo restante: 6s
Tempo restante: 5s
Tempo restante: 4s
Tempo restante: 3s
Tempo restante: 2s
Tempo restante: 1s
Estado: VERDE (Siga)

```



71111L



Sinal Aberto - Siga em frente



Conectar ao Arduino

Simular no Web



Simular Falha



Registro de Eventos

Aguardando atividade...